DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
ASSIGNMENT

Name of the Student	Kailash M
Register Number	192524029
Department	B.Tech – Artificial Intelligence and Data Science (AI&DS)
Subject Code & Subject Name	ECA1012 – Microprocessor and Microcontroller
Title of the Assignment	Analytical Problem Solving Assessment

ASSESSMENT DETAILS

Course Outcome	Develop and execute assembly language programs for microprocessor-based systems, utilizing addressing modes, interrupts, and I/O features in system-level applications.
Duration / Maximum Marks	1 Week / 100
Bloom's Taxonomy Level	BTL6 (K6) – Create / Design.
Comments / Recommendation	

ASSESSMENT RUBRIC

Stage No.	Assessment Criterion	Maximum Marks	Obtained Marks
1	Stage 1 – Interpret the Requirement	10	
2	Stage 2 – Classify and Select Instructions	15	
3	Stage 3 – Design the Operational Logic	15	
4	Stage 4 – Addressing Mode Selection Under Constraints	25	
5	Stage 5 – Develop the 8-bit Assembly Language Program	15	
6	Stage 6 – Test, Verify and Critically Evaluate	20	
TOTAL		**100**	

CODE OF CONDUCT

1. I declare that this assignment is my own work. 2. I have not copied from any other source. 3. I have acknowledged all references used. 4. I will follow academic integrity and SIMATS Engineering ethics.	**STUDENT SIGNATURE** Date: _____	**FACULTY SIGNATURE** Date: _____

ENGINEERING DESIGN REPORT

8086 Software Delay Design: From Clock States to a Verified Delay Routine

ECA1012 – Microprocessor and Microcontroller | CO2 | Kailash M | Reg. No. 192524029

Report flow: Requirement → Clock states → Count → Program → Exact delay → Alternatives → Verification → Decision.

Timing data used (as given in the assignment): MOV register, immediate = 4 states; DEC 16-bit register = 2; NOP = 3; JNZ taken = 16; JNZ not taken = 4; RET = 8. Clock = 10 MHz. The method follows Programs 4.5 and 4.6 of Advanced Microprocessors and Peripherals (Ray and Bhurchandi).

STAGE 1 – Interpret the Requirement

1.1 Time of one clock state

T = 1 / f = 1 / (10 × 10^6 Hz) = 100 ns = **0.1 µs** per clock state. So 10 clock states take 1 µs, and the 100 ms delay corresponds to 1,000,000 clock states; ten minutes (600 s) corresponds to 6,000,000,000 clock states.

1.2 Instructions executed once and on every pass

Instruction	Executed	Role
MOV CX, count	Once per call (once per outer pass in the long delay)	Loads the count; counting starts from this value
DEC CX	Every pass	Reduces the count and sets ZF when CX reaches 0
NOP	Every pass	Adds 3 states of delay; changes no register or flag
JNZ label	Every pass	Jumps back while ZF = 0 (16 states); falls through when ZF = 1 (4 states)
RET	Once per call	Returns to the calling program (8 states)

1.3 Clock states and time of one pass (DEC CX, NOP, JNZ)

Case	Calculation	Clock states	Time
Pass with jump taken	2 + 3 + 16	21	21 × 0.1 µs = **2.1 µs**
Final pass (jump not taken)	2 + 3 + 4	9	0.9 µs

1.4 Largest delay from a single 16-bit count register

The largest count a 16-bit register can start from is 0000H, which DEC CX treats as 65536 (the first DEC gives FFFFH). The delay is then 65536 passes × 2.1 µs = 137.6256 ms (exactly 1,376,256 states, as shown in Stage 3). **The largest single-register delay is about 137.6 ms.**

1.5 Can each delay use one count register?

Delay	Passes needed (approx.)	Fits in 16 bits (≤ 65536)?	Registers
100 ms	100 ms ÷ 2.1 µs ≈ 47,619	Yes (47,619 < 65,536)	One: CX
10 minutes	600 s ÷ 2.1 µs ≈ 285,714,286	No (4,360 times too large for 137.6 ms)	Two: BX and CX (nested loops)

1.6 Addressing mode of each loop instruction

Instruction	Addressing mode	Explanation
MOV CX, 0BA03H	Register (destination) and immediate (source)	The constant is part of the instruction

Instruction	Addressing mode	Explanation
DEC CX	Register	The operand is the register CX
NOP	Implied	No operand
JNZ BACK	Relative (short, 8-bit signed displacement)	Target = next IP + displacement; the displacement is −4 for this loop
RET	Implied (stack)	Return address is popped from the stack

What each delay routine must contain

- **100 ms routine:** a procedure that loads CX with a count of about 47,619, runs DEC CX / NOP / JNZ and ends in RET (one count register).
- **10-minute routine:** a procedure with an inner loop on CX and an outer loop on BX, ending in RET (two count registers).

Delay Requirement Statement

Clock period = 0.1 µs. One pass of the DEC CX–NOP–JNZ loop = 21 states = 2.1 µs. The 100 ms delay needs one count register (CX); the 10-minute delay needs two (BX outer, CX inner).

STAGE 2 – Design the 100 ms Delay Routine

2.1 Instructions selected and their clock states

Instruction	Purpose	Clock states	How often
MOV CX, N	Load count	4	1
DEC CX	Count down, set ZF	2	N
NOP	Timing adjustment	3	N
JNZ BACK	Repeat while CX ≠ 0	16 taken / 4 not taken	N (N − 1 taken, 1 not taken)
RET	Return to caller	8	1

2.2 Expression for the count N

Delay ≈ N × (states per pass) × T, so

> **N = Required delay ÷ (states per pass × clock period) = 100 ms ÷ (21 × 0.1 µs) = 100,000 µs ÷ 2.1 µs = 47,619.05**

N must be a whole number. Candidates: N = 47,619 gives an error of −0.1 µs and N = 47,620 gives +2.0 µs (Stage 3 shows the exact delay is 21N states). **N = 47,619** is chosen because it is the closer one.

2.3 Decimal to hexadecimal conversion and check

Step	Division by 16	Quotient	Remainder (hex digit)
1	47,619 ÷ 16	2,976	3 → 3
2	2,976 ÷ 16	186	0 → 0
3	186 ÷ 16	11	10 → A
4	11 ÷ 16	0	11 → B

Reading the remainders from the last to the first: **47,619 = BA03H** (written 0BA03H in the program because it starts with a letter).

Check (hex back to decimal): BA03H = $11 \times 16^3 + 10 \times 16^2 + 0 \times 16^1 + 3 \times 16^0$ = 45,056 + 2,560 + 0 + 3 = **47,619** ✔️

2.4 Flowchart

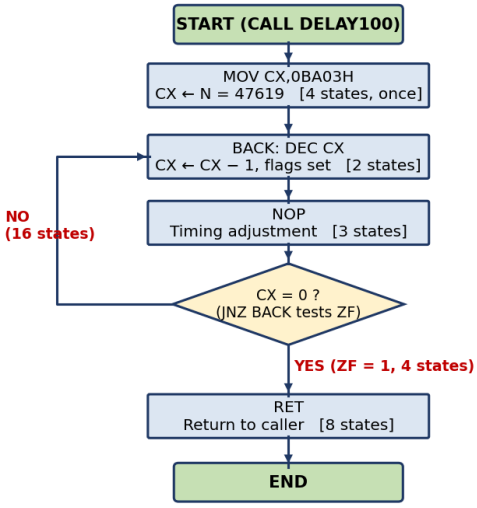


Figure 1: Flowchart of the 100 ms delay routine

2.5 Complete 8086 procedure

Label	Instruction	Comment	Clock states
STK	SEGMENT STACK	Stack segment (needed by CALL and RET)	–
	DW 64 DUP(?)	Reserve 64 words of stack space	–
STK	ENDS	End of stack segment	–
CODE	SEGMENT	Start of the code segment	–
	ASSUME CS:CODE	Tell the assembler that CS refers to CODE	–
START:	CALL DELAY100	Call the 100 ms delay (test call)	–
	HLT	Stop the processor after the delay	–
DELAY100	PROC NEAR	Start of the delay procedure (same-segment call)	–
	MOV CX,0BA03H	CX ← N = 47,619 (BA03H)	4 (once)
BACK:	DEC CX	CX ← CX − 1; sets ZF = 1 when CX = 0	2 (N times)
	NOP	3-state timing adjustment, no effect on CX or flags	3 (N times)
	JNZ BACK	If ZF = 0 jump back to BACK, else fall through	16 / 4
	RET	Return to the instruction after CALL	8 (once)
DELAY100	ENDP	End of the delay procedure	–
CODE	ENDS	End of the code segment	–
	END START	End of program; execution begins at START	–

The segment-register set-up for data (DS) is not needed because the procedure uses no memory data. In emu8086 the stack segment is created automatically; the explicit stack segment above is for MASM/TASM.

2.6 Purpose of the NOP

The NOP does no useful work; it adds **3 clock states** to every pass. With it a pass is 21 states (2.1 µs); without it a pass is 18 states (1.8 µs). It is therefore a timing-adjustment instruction: it lets the programmer set the pass time (and so the count) to a convenient value without touching any register or flag. If it is removed, the pass time falls from 2.1 µs to 1.8 µs, the count for 100 ms rises to N = 55,556

(D904H) and the exact delay becomes 100.0008 ms (Stage 5). The routine still works, but the count and the error both change, and the single-register maximum falls to 65,536 × 1.8 µs = 117.96 ms.

STAGE 3 – Calculate the Exact Delay and the Error

3.1 Number of executions and clock states

Instruction	Times executed	States each	Total states
MOV CX,0BA03H	1	4	4
DEC CX	47,619	2	95,238
NOP	47,619	3	142,857
JNZ BACK (jump taken)	47,618	16	761,888
JNZ BACK (jump not taken)	1	4	4
RET	1	8	8
Total			**999,999**

JNZ is executed N = 47,619 times: it is **taken 47,618 times** (16 states each) and **not taken once** (4 states), on the final pass when CX = 0.

3.2 Exact delay

Exact delay = 999,999 states × 0.1 µs = **99,999.9 µs = 99.9999 ms**. This includes MOV and RET but not the CALL instruction of the caller (not part of the specified routine).

General result: total states = 4 + 21(N − 1) + 9 + 8 = **21N** for any N, which is why N = 47,619 gives 999,999 states.

3.3 Error

Error = 99,999.9 µs − 100,000 µs = **−0.1 µs** (the delay is short by one clock state). As a percentage: 0.1 ÷ 100,000 × 100 = **−0.0001 %**.

3.4 Can the error be removed by one more instruction?

No. The delay is short by exactly **1 clock state** (0.1 µs). The shortest 8086 instruction takes 2 states, so adding it gives 1,000,001 states, which is +0.1 µs: the same size of error with the opposite sign. A NOP (3 states) would give +0.2 µs, which is worse. Because one state cannot be added, the remaining error of 0.1 µs is the best this method can reach at 10 MHz.

Stage 3 Deliverable

Required delay = 100 ms; exact delay = 99.9999 ms (999,999 states); error = −0.1 µs (−0.0001 %).

STAGE 4 – Design the Ten-Minute Delay Routine

4.1 Why two nested loops

One 16-bit register can count at most 65,536 passes, which gives only 137.6 ms. Ten minutes is about 4,360 times longer, so the 137.6 ms inner loop must itself be repeated 4,360 times. The **inner loop is controlled by CX** (counts 65,535 passes) and the **outer loop by BX** (counts how many times the inner loop is repeated); CX is reloaded at the start of every outer pass.

4.2 Time for the inner loop to run to completion (CX = FFFFH = 65,535)

States = 21 × (65,535 − 1) + 9 = 21 × 65,534 + 9 = 1,376,214 + 9 = **1,376,223 states**

Time = 1,376,223 × 0.1 µs = 137,622.3 µs = **137.6223 ms**

4.3 Time for one pass of the outer loop

Part of one outer pass	States
MOV CX,0FFFFH (reload)	4
Inner loop to completion	1,376,223
DEC BX	2
JNZ OUTER (taken)	16
One outer pass	**1,376,245 states = 137.6245 ms**

4.4 Outer count BX for ten minutes

10 minutes = 600 s = 6,000,000,000 states at 0.1 µs.

M = 6,000,000,000 ÷ 1,376,245 = 4,359.69

Candidates: M = 4,359 gives 599.9052 s (−94.80 ms); M = 4,360 gives 600.0428 s (+42.82 ms). **M = 4,360** is closer to ten minutes and is chosen.

Step	Division by 16	Quotient	Remainder (hex digit)
1	4,360 ÷ 16	272	8 → 8
2	272 ÷ 16	17	0 → 0
3	17 ÷ 16	1	1 → 1
4	1 ÷ 16	0	1 → 1

4,360 = 1108H. Check: 1 × 4096 + 1 × 256 + 0 × 16 + 8 = 4,096 + 256 + 0 + 8 = **4,360** ✔️. The inner count FFFFH = 15 × 4096 + 15 × 256 + 15 × 16 + 15 = 65,535 ✔️.

4.5 Flowchart of the nested-loop routine

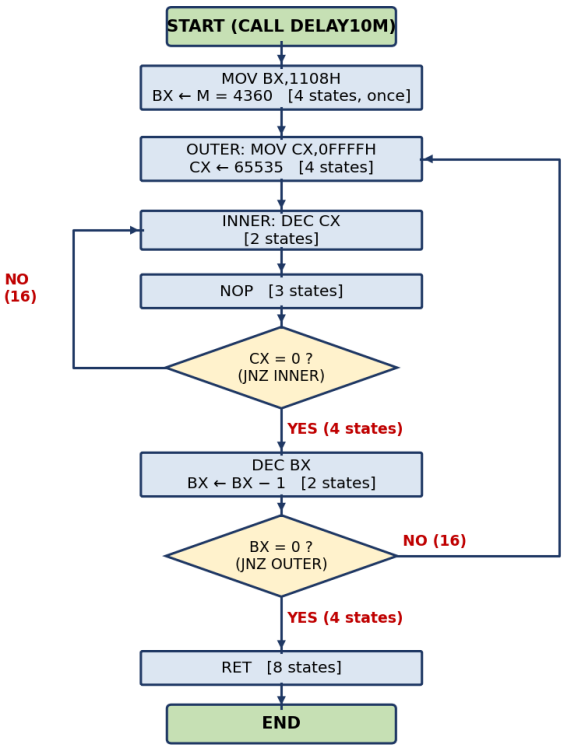


Figure 2: Flowchart of the ten-minute (nested loop) delay routine

4.6 Complete 8086 procedure

Label	Instruction	Comment	Clock states
STK	SEGMENT STACK	Stack segment (needed by CALL and RET)	–
	DW 64 DUP(?)	Reserve 64 words of stack space	–
STK	ENDS	End of stack segment	–
CODE	SEGMENT	Start of the code segment	–
	ASSUME CS:CODE	CS refers to the CODE segment	–
START:	CALL DELAY10M	Call the ten-minute delay (test call)	–
	HLT	Stop after the delay	–
DELAY10M	PROC NEAR	Start of the ten-minute delay procedure	–
	MOV BX,1108H	BX ← outer count M = 4,360 (1108H)	4 (once)
OUTER:	MOV CX,0FFFFH	CX ← inner count 65,535 (FFFFH) on every outer pass	4 (M times)
INNER:	DEC CX	CX ← CX − 1; ZF = 1 when CX = 0	2
	NOP	3-state timing adjustment in the inner loop	3
	JNZ INNER	Repeat inner loop while CX ≠ 0	16 / 4
	DEC BX	BX ← BX − 1; ZF = 1 when BX = 0	2 (M times)
	JNZ OUTER	Repeat outer loop while BX ≠ 0	16 / 4
	RET	Return to the caller	8 (once)
DELAY10M	ENDP	End of procedure	–
CODE	ENDS	End of the code segment	–
	END START	End of program; entry point START	–

4.7 Exact delay and error

Instruction	Times executed	States each	Total states
MOV BX,1108H	1	4	4
MOV CX,0FFFFH	4,360	4	17,440
DEC CX	285,732,600	2	571,465,200
NOP	285,732,600	3	857,197,800
JNZ INNER (taken)	285,728,240	16	4,571,651,840
JNZ INNER (not taken)	4,360	4	17,440
DEC BX	4,360	2	8,720
JNZ OUTER (taken)	4,359	16	69,744
JNZ OUTER (not taken)	1	4	4
RET	1	8	8
Total			**6,000,428,200**

Exact delay = 6,000,428,200 × 0.1 µs = 600,042,820 µs = **600.04282 s** (10 min 0.0428 s).

Error = 600.04282 s − 600 s = **+42.82 ms**, which is 42.82 ms ÷ 600,000 ms × 100 = **+0.0071 %**.

Check by formula: total states = M × (inner + 22) = 4,360 × (1,376,223 + 22) = 4,360 × 1,376,245 = 6,000,428,200 ✔️ (the MOV BX, the last JNZ not taken (−12) and RET add 4 − 12 + 8 = 0, so they cancel).

4.8 Longest delay of the two-register routine

The longest delay occurs with BX = 0000H (65,536 outer passes) and CX = 0000H (65,536 inner passes each).

- Inner loop = 21 × 65,535 + 9 = 1,376,244 states.
- One outer pass = 4 + 1,376,244 + 2 + 16 = 1,376,266 states.
- Total = 65,536 × 1,376,266 = **90,194,968,576 states = 9,019.4969 s ≈ 150.3 minutes (about 2 h 30 min)**.

STAGE 5 – Evaluate the Alternatives

5.1 Same system on a 5 MHz clock

T = 0.2 µs; one pass = 21 × 0.2 = 4.2 µs; the exact delay formula is 21N states × 0.2 µs.

- **100 ms:** N = 100,000 µs ÷ 4.2 µs = 23,809.5. N = 23,810 (5D02H) gives 500,010 states = 100.002 ms (+2.0 µs); N = 23,809 gives −2.2 µs. Choose **23,810 = 5D02H**.
- **Ten minutes:** one outer pass = 1,376,245 states = 275.249 ms. M = 600 ÷ 0.275249 = 2,179.8, so **M = 2,180 = 0884H** (inner count still FFFFH). Total = 2,180 × 1,376,245 = 3,000,214,100 states × 0.2 µs = 600.04282 s (+42.82 ms, as before).
- Both counts fit in 16 bits and get smaller because each state is twice as long.

5.2 Loop written without the NOP

Pass = 2 + 16 = 18 states = 1.8 µs. Total states = 4 + 18(N − 1) + 6 + 8 = 18N.

- **100 ms:** N = 100,000 ÷ 1.8 = 55,555.6. N = 55,556 (**D904H**) gives 1,000,008 states = 100.0008 ms (+0.8 µs); N = 55,555 gives −1.0 µs.
- **Ten minutes:** inner (CX = FFFFH) = 18 × 65,534 + 6 = 1,179,618 states; one outer pass = 4 + 1,179,618 + 2 + 16 = 1,179,640 states. M = 6 × 10^9 ÷ 1,179,640 = 5,086.2, so **M = 5,086 (13DEH)**, giving 599.9649 s (−35.10 ms). M = 5,087 would give +82.87 ms.

5.3 Ten minutes by calling the 100 ms procedure repeatedly

10 min ÷ 100 ms = 6,000 calls. The count 6,000 = **1770H** (1 × 4096 + 7 × 256 + 7 × 16 + 0 = 6,000 ✔️) fits easily in BX. Code: MOV BX,1770H; AGAIN: CALL DELAY100; DEC BX; JNZ AGAIN.

States = 4 + 6,000 × (19 + 999,999 + 2) + 5,999 × 16 + 4 = 6,000,215,992 → **600.0216 s (+21.60 ms, +0.0036 %)**. CALL near = 19 states is the standard 8086 data-sheet value (not given in the assignment). The 100 ms routine is shorter by 0.1 µs per call, but the CALL, DEC BX and JNZ overhead adds about 3.7 µs per call, so the net error is positive.

5.4 Programmable timer with an interrupt

An 8254 programmable interval timer fed with a 1 MHz clock: Counter 0 in mode 2 divides by 10,000 (2710H) to give 100 pulses per second (10 ms); Counter 1 in mode 0 (interrupt on terminal count), clocked by the output of Counter 0, is loaded with 60,000 (EA60H; 14 × 4096 + 10 × 256 + 6 × 16 + 0 = 60,000 ✔️) and raises one interrupt after 60,000 × 10 ms = 600 s. The interrupt reaches the 8086 through an 8259 interrupt controller. Control words: Counter 0 = 34H, Counter 1 = 70H. Both counts fit in 16 bits.

5.5 Comparison

Criterion	Nested loops (10 MHz, baseline)	5 MHz clock	No NOP	100 ms called 6,000 times	Timer + interrupt
Counts (hex) and 16-bit fit	BX = 1108H, CX = FFFFH; fit	BX = 0884H, CX = FFFFH; fit	BX = 13DEH, CX = FFFFH; fit	BX = 1770H, CX = BA03H; fit	2710H and EA60H; fit
Accuracy of the delay	+42.82 ms (+0.0071 %)	+42.82 ms (+0.0071 %); 100 ms: +2 µs	−35.10 ms (−0.0058 %); 100 ms: +0.8 µs	+21.60 ms (+0.0036 %)	Set by the timer clock (crystal) only, e.g. ±100 ppm = ±60 ms
Effect of an interrupt during the delay	Each interrupt lengthens the delay by its service time	Same	Same	Same	None: the timer keeps counting during the ISR

Criterion	Nested loops (10 MHz, baseline)	5 MHz clock	No NOP	100 ms called 6,000 times	Timer + interrupt
Processor free for other work?	No, busy for 10 min	No	No	No (the main loop could do work between calls, but the timing then drifts)	Yes, free until the interrupt
Extra hardware	None	None	None	None	8254 timer, 8259 controller, address decoding
Ease of changing the delay	Recalculate BX and CX	Recalculate	Recalculate	Change BX only	Reprogram counter values
Dependence on 8086 clock	Delay ∝ 1 / clock	Counts must be redone	Counts must be redone	∝ 1 / clock	Independent of the 8086 clock

Design Decision

I recommend a programmable timer (8254) with an interrupt for the ten-minute delay because it leaves the 8086 free to run the process controller during the ten-minute hold time, its accuracy depends only on the timer clock and is not changed by interrupts or by the instruction mix, and the delay is changed by reprogramming a count instead of recalculating loops.

Why the alternatives were not selected:

- **Nested software loop (10 MHz or 5 MHz):** accurate to 0.007 %, but the processor does nothing for ten minutes, any interrupt adds its service time to the delay, and the counts must be recalculated if the clock changes.
- **Loop without NOP:** gives no advantage in accuracy (−35.1 ms against +42.8 ms) and has the same drawbacks of busy waiting.
- **Calling the 100 ms procedure 6,000 times:** simplest to change and slightly more accurate here (+21.6 ms), but it still keeps the CPU busy and the error depends on the CALL overhead.

The 100 ms display delay stays a software loop: its error is only 0.1 µs, it needs no extra hardware, and 100 ms of waiting is acceptable between display outputs.

STAGE 6 – Verify and Critically Evaluate the Design

*The tables below give the **predicted values calculated by hand**. The "Observed" columns and screenshot boxes are to be filled with the readings from emu8086 (or MASM/TASM with DEBUG) after single-stepping the programs.*

6.1 Loop operation (first three passes, N = 47,619)

Pass	Instruction	CX after DEC CX (predicted)	ZF	JNZ	Observed CX / ZF
Start	MOV CX,0BA03H	BA03H (47,619)	–	–	
1	DEC CX	BA02H (47,618)	0	Taken (16 states)	
2	DEC CX	BA01H (47,617)	0	Taken	
3	DEC CX	BA00H (47,616)	0	Taken	
47,619 (last)	DEC CX	0000H (0)	1	Not taken (4 states), RET executes	

JNZ stops jumping on pass 47,619, when DEC CX brings CX from 0001H to 0000H and sets ZF = 1.

[Insert emu8086 screenshot: single-step of DELAY100 showing CX = BA02H, BA01H, BA00H and ZF after each DEC CX]

6.2 Reduced-count tests

Short delay with CX = 0005H (change the immediate to 0005H). Predicted: 17 instructions executed, 105 states = 10.5 µs.

Instruction	Predicted executions	States	Observed
MOV CX,0005H	1	4	
DEC CX	5	10	
NOP	5	15	
JNZ taken / not taken	4 / 1	64 + 4	
RET	1	8	
Total	**17**	**105**	

Long delay with BX = 0003H and CX = 0004H. Inner loop = 21 × 3 + 9 = 72 states. One outer pass = 4 + 72 + 2 + 16 = 94 states. Total = 3 × 94 = 282 states (formula M × (inner + 22) = 3 × 94) = 28.2 µs. Predicted: 47 instructions.

Instruction	Predicted executions	States	Observed
MOV BX,0003H	1	4	
MOV CX,0004H	3	12	
DEC CX	12	24	
NOP	12	36	
JNZ INNER taken / not taken	9 / 3	144 + 12	
DEC BX	3	6	
JNZ OUTER taken / not taken	2 / 1	32 + 4	
RET	1	8	
Total	**47**	**282**	

[Insert emu8086 screenshot: instruction counts for the reduced-count runs]

6.3 Count values in the assembled listing

Expected machine code (immediate operands are stored low byte first):

Instruction	Expected machine code	Count (hex)	Agrees with
MOV CX,0BA03H	B9 03 BA	BA03H = 47,619	Stage 2
MOV BX,1108H	BB 08 11	1108H = 4,360	Stage 4
MOV CX,0FFFFH	B9 FF FF	FFFFH = 65,535	Stage 4
DEC CX / NOP / DEC BX / RET	49 / 90 / 4B / C3	–	–
JNZ BACK / JNZ INNER / JNZ OUTER	75 FC / 75 FC / 75 F6	Displacement −4, −4, −10	Short relative jumps

[Insert emu8086 screenshot: assembled listing showing B9 03 BA, BB 08 11 and B9 FF FF]

6.4 Use of the delay: square wave on an output port

Label	Instruction	Comment	Clock states
`AGAIN:`	`MOV AL,01H`	AL ← 01H (bit D0 = 1)	4

Label	Instruction	Comment	Clock states
	`OUT 0C0H,AL`	Set the output bit (port 0C0H assumed)	10
	`CALL DELAY100`	Hold the bit high for 100 ms	19 + 999,999
	`MOV AL,00H`	AL ← 00H (bit D0 = 0)	4
	`OUT 0C0H,AL`	Clear the output bit	10
	`CALL DELAY100`	Hold the bit low for 100 ms	19 + 999,999
	`JMP AGAIN`	Repeat for a continuous square wave	15

The states for MOV AL,imm (4), OUT port,AL (10), CALL near (19) and JMP short (15) are standard 8086 data-sheet values, since the assignment does not list them.

- High time = 4 + 10 + 19 + 999,999 = 1,000,032 states = 100.0032 ms.
- Low time = 1,000,032 + 15 (JMP) = 1,000,047 states = 100.0047 ms.
- Period = 2,000,079 states = 200.0079 ms, so **f = 1 ÷ 0.2000079 s ≈ 4.9998 Hz (≈ 5 Hz)**.
- Effect of the instructions outside the delay: they add 81 states (8.1 µs) per cycle, i.e. 0.004 % of the period, so the frequency is lowered from the ideal 5.0000 Hz by about 0.2 mHz and the duty cycle is not exactly 50 % (by 1.5 µs). This is negligible for a display, but it shows that the delay measured at the output is always the delay routine plus the surrounding instructions.

[Insert emu8086 screenshot: output-port bit toggling / logic-analyser view or register window during the square-wave test]

6.5 Specification verification

Requirement	Specified	Achieved	Satisfied?
Short delay	100 ms	99.9999 ms (999,999 states)	Yes
Count for short delay (hexadecimal)	–	CX = BA03H (47,619)	Yes
Error in short delay	As small as possible	−0.1 µs (−0.0001 %)	Yes
Long delay	10 minutes	600.04282 s (6,000,428,200 states)	Yes
Outer and inner counts (hexadecimal)	–	BX = 1108H (4,360), CX = FFFFH (65,535)	Yes
Error in long delay	As small as possible	+42.82 ms (+0.0071 %)	Yes (within 0.01 %)
Procedure form	PROC … ENDP ending in RET	DELAY100 and DELAY10M	Yes
Count registers	CX short; BX and CX long	As specified	Yes

FINAL ENGINEERING DECISION

Would you use these delay routines in the controller? NO for the ten-minute software routine; YES for the 100 ms routine.

I would keep the 100 ms software delay and replace the ten-minute software delay by a timer-and-interrupt design.

Criterion	Evidence from the design
Accuracy of the two delays	100 ms routine: −0.1 µs (−0.0001 %). Ten-minute routine: +42.82 ms (+0.0071 %). Both are accurate enough for a process hold time.
Dependence on clock frequency	Delay = 21N × T for the short routine and M × 1,376,245 × T for the long one; at 5 MHz every count must be recalculated (23,810 and 2,180).

Criterion	Evidence from the design
Effect of interrupts	A software loop does not count time, so each interrupt adds its service time to the delay; the timer keeps counting.
Processor time spent doing nothing	The 8086 is idle for the full 600 s (about 6×10^9 clock states) in the software version, and for 100 ms in the short one.
Hardware cost of the alternative	An 8254 and 8259 with address decoding: a few low-cost chips, justified for a ten-minute wait but not for 100 ms.
Ease of changing the delay	Software: recalculate counts and error by hand. Timer: reload 2710H and EA60H.

Redesign for the ten-minute delay

- **Change:** remove DELAY10M. Program the 8254: Counter 0 mode 2 with count 2710H (10,000) on a 1 MHz clock, Counter 1 mode 0 with count EA60H (60,000), and connect its output to an 8259 interrupt input; write an interrupt service routine that sets a “hold finished” flag.
- **Why:** it frees the 8086 for the whole ten minutes, removes the dependence of the delay on the 8086 clock, and the delay is no longer lengthened by other interrupts.
- **Predicted effect:** delay accuracy equal to the timer-clock accuracy (for example ±100 ppm = ±60 ms over 600 s, similar in size to the present software error but fixed and independent of the program); the rest of the program is unchanged except that it tests the flag instead of calling DELAY10M. The 100 ms delay and its square-wave use stay as they are.

References

- Ray, A. K. and Bhurchandi, K. M., Advanced Microprocessors and Peripherals, McGraw-Hill (Programs 4.5 and 4.6).
- Intel Corporation, 8086 Family User’s Manual and 8254 Programmable Interval Timer data sheet.